

# Configure CI Pipeline as Code with YAML

In this exercise, you will configure CI Pipeline as code with YAML.

Task 1: Import the YAML build definition for CI

In this task, you will add the YAML build definition that will be used to implement the Continuous Integration.

Let's start by importing the CI pipeline named [eshoponweb-ci.yml](#).

1. Go to **Pipelines > Pipelines**.
2. Click on **New Pipeline** button.
3. Select **Azure Repos Git (YAML)**.
4. Select the **eShopOnWeb** repository.
5. Select **Existing Azure Pipelines YAML File**.
6. Select the **main** branch and the **/.ado/eshoponweb-ci.yml** file, then click on **Continue**.

The CI definition consists of the following tasks:

- **DotNet Restore:** With NuGet Package Restore you can install all your project's dependency without having to store them in source control.
  - **DotNet Build:** Builds a project and all of its dependencies.
  - **DotNet Test:** .Net test driver used to execute unit tests.
  - **DotNet Publish:** Publishes the application and its dependencies to a folder for deployment to a hosting system. In this case, it's **Build.ArtifactStagingDirectory**.
  - **Publish Artifact - Website:** Publish the app artifact (created in the previous step) and make it available as a pipeline artifact.
  - **Publish Artifact - Bicep:** Publish the infrastructure artifact (Bicep file) and make it available as a pipeline artifact.
7. Click on **Run** and wait for the pipeline to execute successfully.

## Task 2: Enable Continuous Integration

The default build pipeline definition doesn't enable Continuous Integration.

1. Click on the **Edit pipeline** option under the ellipsis menu near **Run new** button in the top-right
2. Now, you need to replace the **# trigger:** and **# - main** lines with the following code:

code

trigger:

branches:

include:

- main

paths:

include:

- src/web/\*

This will automatically trigger the build pipeline if any change is made to the main branch and the web application code (the src/web folder).

Since you enabled Branch Policies, you need to pass by a Pull Request in order to update your code.

3. Click the **Validate and save** button to validate and save the pipeline definition.
4. Select **Create a new branch for this commit**.
5. Keep the default branch name and **Start a pull request** checked.
6. Click on **Save**.
7. Your pipeline will take a name based on the project name. Let's **rename** it for identifying the pipeline better. Go to **Pipelines > Pipelines** and click on the recently created pipeline. Click on the ellipsis and **Rename/Move** option. Name it **eshoponweb-ci** and click on **Save**.
8. Go to **Repos > Pull Requests**.
9. Click on the **"Update eshoponweb-ci.yml for Azure Pipelines"** pull request.
10. After all validations are successful, on the top-right click on **Approve**. Now you can click on **Complete**.
11. On the **Complete Pull Request** tab, Click on **Complete Merge**

### Task 3: Test the CI pipeline

In this task, you will create a Pull Request, using a new branch to merge a change into the protected **main** branch and automatically trigger the CI pipeline.

1. Navigate to the **Repos** section, and click on **Branches**.
2. Create a new branch named **Feature02** based on the **main** branch.
3. Click the new **Feature02** branch.
4. Navigate to the **/eShopOnWeb/src/Web/Program.cs** file and click **Edit** in the top-right.
5. Remove the first line:

// Testing my PR

6. Click on **Commit > Commit** (leave default commit message).
7. A message will pop-up, proposing to create a Pull Request (as your **Feature02** branch is now ahead in changes, compared to **main**).
8. Click on **Create a Pull Request**.
9. In the **New pull request** tab, leave defaults and click on **Create**.
10. The Pull Request will show some pending requirements, based on the policies applied to the target **main** branch.
11. After all validations are successful, on the top-right click on **Approve**. Now from the **Set auto-complete** dropdown you can click on **Complete**.
12. On the **Complete Pull Request** tab, Click on **Complete Merge**
13. Go back to **Pipelines > Pipelines**, you will notice that the build **eshoponweb-ci** was triggered automatically after the code was merged.
14. Click on the **eshoponweb-ci** build then select the last run.
15. After its successful execution, click on **Related > Published** to check the published artifacts:
  - Bicep: the infrastructure artifact.
  - Website: the app artifact.